



13

Server-Side Rendering

As we discussed in *Chapter 1, Why React?*, the React library is remarkably flexible in terms of how our components can be transformed into various target formats. One such target format, as you might have guessed, is standard HTML markup, presented as a string and generated on the server. In this chapter, we will delve into how **server-side rendering (SSR)** works in React and the advantages it offers both users and developers. You will learn why this approach can be valuable for your application and how it enhances the overall user experience and performance.

The following topics are covered in this chapter:

- Working on the server
- Using Next.js
- React Server Components

Technical requirements

You can find the code files of this chapter on GitHub at

<https://github.com/PacktPublishing/React-and-React-Native-5E/tree/main/Chapter13>.

Working on the server

Web technologies have come a long way or, more precisely, have come full circle. It all started with static web pages prepared by a server. Servers were the foundation of all website and application logic, as they were entirely responsible for their functioning. Then, we tried to move away from **SSR** in favor of rendering pages in the browser, which led to a significant leap in the development of web pages as fully fledged applications, now comparable to desktop ones. As a result, browsers became the core of application logic, while servers merely provided data for applications.

Currently, the development cycle has brought us back to SSR and server components, but now we have a unified logic and code for both the server and the client. Why this happened and what conclusions and experiences we have gained with the evolution of technologies are what we will try to understand in this section, and we will simultaneously learn about the types of work our applications do on the server.

Server-side rendering

In a traditional **single-page application (SPA)** approach, we rely solely on local browser rendering. We write all the code, styles, and markup specifically for a browser, and during the application build, we get static HTML, CSS, and JavaScript files, which are then loaded into the browser.

In most cases, the initial HTML file remains empty, without any content. The only important thing to have in this file is the connected JavaScript file, which will render everything we need.

Below is a schematic illustration of how an SPA application is loaded and rendered:

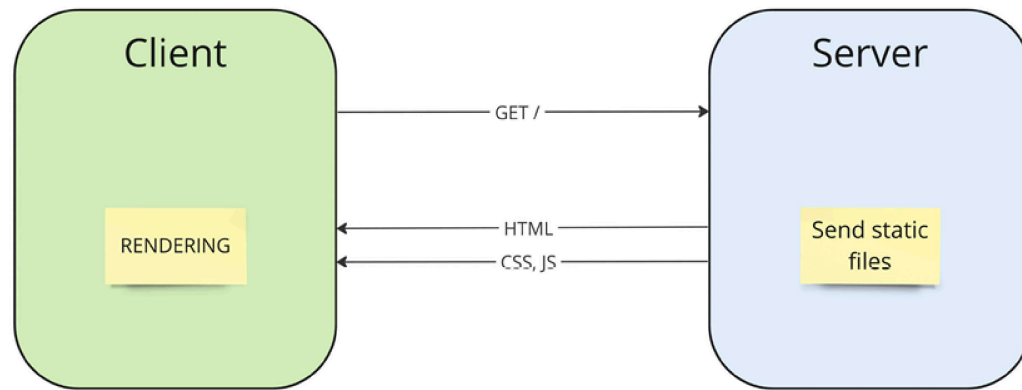


Figure 13.1: SPA application

This approach brought interactivity, making applications feel and function like real desktop applications. There's no longer a need to reload a page every time to update the content and receive notifications, new emails, or messages, as the entire application logic is directly in the browser. Over time, browser applications have almost entirely replaced desktop ones. Now, we can write emails, work with documents, watch movies, and do much more, all within a single browser. Many companies, instead of developing desktop applications, started creating their projects as web applications. The browser's ability to operate across any architecture and operating system significantly reduced development costs.

At the same time, servers also underwent changes, moving away from page templating, caching, etc. Backend developers no longer need to fo-

cus on page layout and can devote more time to more complex logic and architecture.

However, SPA applications do have drawbacks, including the long initial load time due to the need to download and process the script. During this process, the user sees a white screen or a loading spinner. Additionally, the empty initial HTML file is not suitable for search engine optimization, as search engines perceive it as a blank page.

In the context of creating, for example, an online store, a regular React SPA might not be suitable because it's important for users and search engines to immediately see the page content. Before the advent of SPAs, such tasks were solved by tools that worked only on the server side, which always prepared the content. In React, solving this problem is more complex because, as we know, React works on the browser side.

The first step in the solution will obviously be the idea of rendering the page content on the server by React. And this won't be a problem. Since its release, React has had the `renderToString` function for this purpose, which can be called in a **Node.js server** environment. This function returns an HTML string that, when sent to the browser, allows the content to be rendered on the user's screen.

Let's see how SSR with the `renderToString` function would work:

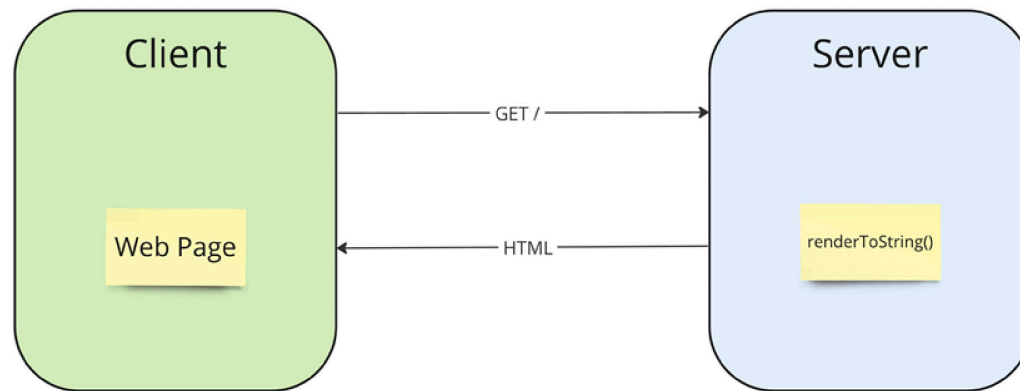


Figure 13.2: Server rendering using `renderToString`

In this example, when a page is requested in the browser, the server, by calling the function `renderToString` and passing it to the React component tree, outputs HTML. By sending this HTML string in response to the browser's request, the browser renders the result.

However, in such an example, the HTML generated on the server and rendered in the browser lacks interactivity and the capabilities of a client application. For functionalities like buttons, navigation, and everything we are accustomed to in SPAs, JavaScript is required. Therefore, the next step in implementing an interactive site or application rendered on the server is to transmit not only HTML but also JavaScript, which will provide all the interactivity we need.

To solve this problem, the approach of **isomorphic JavaScript** was introduced. Code written in this style can be executed first on the server and then on the client. This allows you to prepare the initial render on the server and send the ready HTML along with the JavaScript bundle to the

client, allowing the browser to then provide interactivity. This approach speeds up the initial load of the application, while maintaining its functionality and allowing search engines to index the page in search results.

When a user opens a page, they immediately see the rendering result performed on the server even before the JavaScript is loaded. This quick initial response significantly improves the user experience. After the page and JS bundle are loaded, it's crucial for the browser to **hydrate** the page, as we know from the `renderToString` example that all our elements lack interactivity. For this, the script needs to attach all necessary event listeners to the elements. This process is called **hydration** and is a lighter and faster process compared to a full-page render from scratch.

Another important feature of interactivity is the ability to navigate through an application instantly or smoothly without reloading a browser page. With isomorphic JavaScript, this became possible, as it is enough to load the JavaScript code of the next page, and the application can then render the next page locally.

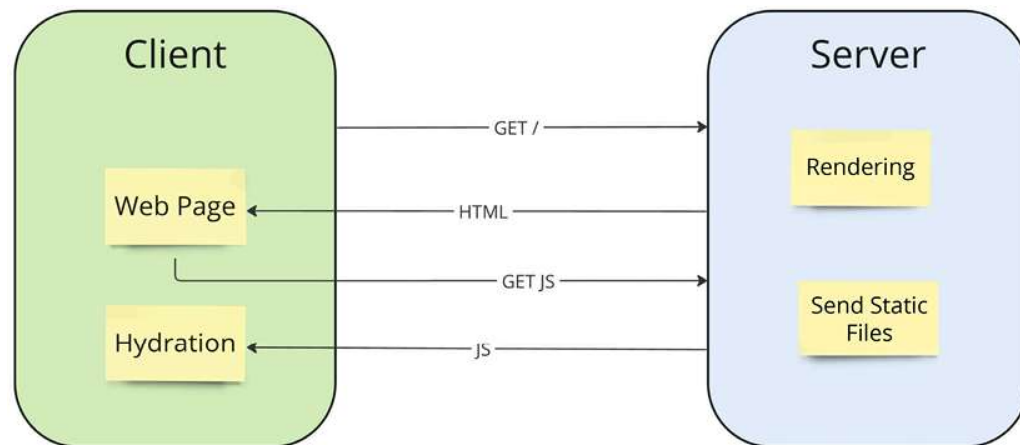


Figure 13.3: SSR

The figure above schematically represents the SSR approach, where the application is fully interactive. Initially, when a page is requested, the server renders the content and returns HTML with an attached JavaScript bundle. Then, the browser loads the JS file and hydrates all the content previously displayed on the page. This approach is what has come to be known as SSR. It has become widely used among React developers and has found its place in the arsenal of modern web technologies. SSR combines the fast loading of page content and the high performance of server rendering with the flexibility and interactivity of a client application.

Static site and incremental static generation

Although SSR represents a significant improvement, it is not a universal solution and has its drawbacks, including the need to generate a page from scratch for each request. For example, pages that do not have dynamic content must be generated on a server each time, which can cause display delays for the user. Additionally, even for the simplest applications or sites, SSR requires a Node.js server for rendering, unlike SPAs where it was sufficient to use a **content delivery network (CDN)** to position application files closer to the user, thereby speeding up loading.

The solution to these problems was found in the **static site generation (SSG)** approach. The logic of SSG is to render all static pages on a server during the project build process. As a result, we get many HTML pages ready for immediate delivery upon request. As with SSR, in SSG, the JavaScript bundle hydrates the page after it has loaded, making it inter-

active. In the end, we get the same experience as with SPAs but not with an empty HTML file: rather, one full of content for quick rendering. SSG projects can be hosted on fast web servers or CDNs, which also allows for additional caching and speeding up the loading time of such applications.

SSG became an ideal solution for websites, blogs, and simple online stores, ensuring fast page load times without blocking requests, SEO support, and the same interactivity as SPAs. Moreover, it became possible to combine SSR for dynamic data and SSG for static pages. This hybrid approach opens up new possibilities to implement more complex projects, combining the advantages of both methods. It allows developers to optimize performance and user experience by choosing the best rendering method, depending on the specific requirements of each page of the site or application.

Another issue faced by developers and companies is updating statically generated pages. For example, traditionally, adding a new blog post or updating an online store's inventory required a complete rebuild of a project, which can be time-consuming and inconvenient, especially in large projects. Imagine a blog with 1,000 posts having to be completely rebuilt and re-rendered just because a new post being added.

This problem is solved by an approach known as **incremental static generation (ISR)**. ISR combines the principles of SSG and SSR with caching functionality. To understand this approach, imagine all our generated HTML and JS files at the build stage simply as a **cache**, representing the current result of the project build. As with any cache, we now need to introduce a logic for its revalidation. As long as our cache is

valid, all page requests work as before using the SSG approach. But when the revalidation time expires, the next request to the page initiates its re-rendering on the server in SSR mode. The resulting output is sent to the client and simultaneously replaces the old HTML file with a new one, i.e., updates the cache. The application then continues to operate in SSG mode.

Thanks to ISR, it became possible to implement large-scale projects with millions of pages, which do not need to be constantly rebuilt for minor updates. It also became possible to altogether skip the generation of pages at the build stage, as the required pages would be rendered and saved upon request. For huge projects, this provided a significant increase in project build speeds.

Currently, SSG with ISR, combined with traditional SSR, is one of the most popular approaches for implementing both simple websites and blogs, as well as complex applications. However, traditional SPAs remain a very popular solution. But if we know how to create and assemble SPAs, what about everything else we just discussed? In response to this question, it's important to note that you don't need to develop all these approaches manually. There are several frameworks based on React that provide all the functionalities described above:

- **Next.js:** This framework is known for its flexibility and powerful features. Next.js started with SSR but now supports both SSR and SSG, including ISR support. Recently, Next.js has been working extensively on a new concept for implementing applications using server components, which we will discuss at the end of the chapter.

- **Gatsby:** Gatsby's main distinction is its strong focus on generating static sites using data from various sources (such as **CMS** or **Markdown**). While there are not as many differences from Next.js as there used to be, it remains a fairly popular solution.
- **Remix:** This is a relatively new framework that focuses on closer integration with web standards and improving the user experience. Remix offers unique approaches to data handling and routing, where we can work not page by page but by sections of a page, implementing nested navigation by changing and caching only the part of the page that requires dynamics.

All these frameworks collectively provide a similar experience and implementation of the approaches we've discussed. Next, we'll explore how to implement SSR and static generation using Next.js.

Using Next.js

After familiarizing ourselves with the theory of SSR, let's see how we can implement all this in practice using the **Next.js** framework.

Next.js is a popular React-based framework specifically designed to simplify the process of SSR and static site generation. It offers powerful and flexible capabilities for creating high-performance web applications.

The features of Next.js:

- **An easy-to-use API that automates SSR and static generation:** You just need to write code using the provided methods and functions, and the framework will automatically determine which pages should

be rendered server-side and which can be rendered during the project build process.

- **File-based routing:** Next.js uses a simple and intuitive routing system based on the folder and file structure in the project. This greatly simplifies the creation and management of routes in the application.
- The ability to create comprehensive full-stack applications, thanks to API routes that allow you to implement server-side REST API endpoints.
- The optimization of images, fonts, and scripts, enhancing the performance of projects.

Another important feature of the framework is its close collaboration with the React Core team on implementing new React features. As a result, Next.js currently supports two types of application implementations, known as the **Pages Router** and the **App Router**. The former implements the main functionality we discussed earlier, while the latter is a newer approach designed for working with React Server Components. We will examine the new approach later in this chapter, but for now, let's start with the Pages Router.

To start working with Next.js, you only need to execute a single command that will set everything up for you:

```
npx create-next-app@latest
```

This CLI command will ask you a few questions:

```
✓ What is your project named? ... using-nextjs
✓ Would you like to use TypeScript? ... No / Yes
✓ Would you like to use ESLint? ... No / Yes
✓ Would you like to use Tailwind CSS? ... No / Yes
✓ Would you like to use `src/` directory? ... No / Yes
✓ Would you like to use App Router? (recommended) ... No / Yes
✓ Would you like to customize the default import alias (@/*)? ... No / Yes
✓ What import alias would you like configured? ... @/* No / Yes
```

For our current example, you should answer `Yes` to all questions except for the one about using the App Router. Also, you can access the ready-made example that we'll discuss further at the provided link:

<https://github.com/PacktPublishing/React-and-React-Native-5E/tree/main/Chapter13/using-nextjs>.

In the example, we will create a small website with multiple pages, each using different server rendering approaches. In Next.js, each page of the website should be placed in separate files with names that correspond to the URL path. In our project example:

- The main page of the website, accessible at the root path `domain.com/`, will be located in the `index.tsx` file in the `pages` folder. For the understanding of the following examples, the path to this file, in the case of the main page, will be `pages/index.tsx`.
- The `/about` page will be located in the `pages/about.tsx` file.
- Next, we will create a `/posts` page at the path `pages/posts/index.tsx`.
- Each individual post page will be located in a file using the path `pages/posts/[post].tsx`. Files with names in square brackets indi-

cate to Next.js that this will be a dynamic page, with the `post` variable as a parameter. This means that pages like `/posts/1` and `/posts/2` will use this file as the page component.

- This is how the file routing works. The main directory of the project is the `pages` folder, where we can nest files that will be used to generate website pages based on the structure and names of files and folders.

In the `pages` folder, there are also two service files that are not actual pages but are used by the framework to prepare pages:

- The `_document.tsx` file is necessary for preparing the HTML markup. In this file, we have access to the `<html>` and `<body>` tags. This file is always rendered on the server.
- The `_app.tsx` file is used to initialize the page. You can use this component to connect scripts or for the root layout of pages that will be reused between routes.

Let's add a header to our website in the `App` component. Here's how the `_app.tsx` file looks:

```
const inter = Inter({ subsets: ["latin"] });
export default function App({ Component, pageProps }: AppProps) {
  return (
    <div className={inter.className}>
      <header className="p-4 flex items-center gap-4">
        <Link href="/">Home</Link>
        <Link href="/posts">Posts</Link>
        <Link href="/about">About</Link>
      </header>
```

```
    <div className="p-4">
      <Component {...pageProps} />
    </div>
  </div>
);
}
```

The `App` component returns markup that will be used on every page of our project, which means we will see this header on any of our pages. Additionally, we can use the component control where the rest of the dynamic part of the project will be located.

Now, let's take a look at how the main page of our project will look:

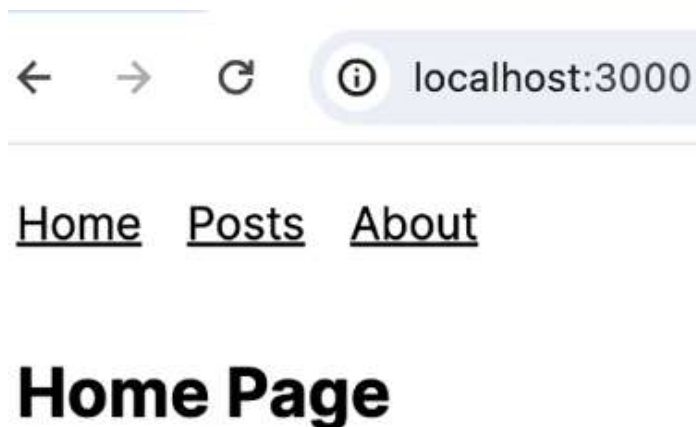


Figure 13.4: Home Page

On this page, we can see the website header with links and the title, which was taken from the `pages/index.tsx` file:

```
export default function Home() {  
  return (  
    <main>  
      <h1>Home Page</h1>  
    </main>  
  );  
}
```

The `pages/index.tsx` file exports only one component with a title inside. It's important to note that this page doesn't have any additional functions or parameters and will be automatically rendered during the project build process. This means that when we visit this page, we get ready-made HTML that the browser can render immediately.

We can confirm that by visiting `localhost:3000/`, we receive the prepared markup. To do this, we just need to open the browser's developer tools and inspect what is returned for this request.

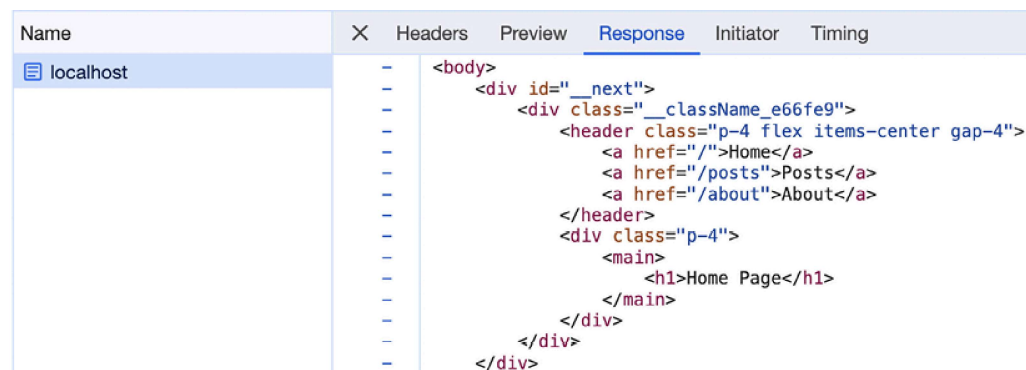


Figure 13.5: Home page response in Chrome DevTools

We can see how Next.js has taken content from the `App` and `Home` components and assembled HTML from it. All of this was done on the server side, not in the browser.

Next, let's take a look at the `/about` page. On this page, we will implement SSR, which means that instead of generating HTML during the build, the page will be rendered on every request. For this purpose, Next.js provides the `getServerSideProps` function, which runs at the time of the page request and returns props used by the component for rendering.

For our example, I've taken some logic from *Chapter 11, Fetching Data from a Server*, where we fetched user data from GitHub. Let's see what the `about.tsx` file will look like:

```
export const getServerSideProps = (async () => {  
  const res = await fetch("https://api.github.com/users/sakhnyuk");  
  const user: GitHubUser = await res.json();  
  return { props: { user } };  
}) satisfies GetServerSideProps<{ user: GitHubUser }>;
```

In the `getServerSideProps` function, we request user data using the **Fetch API**. The data we receive is stored in the `user` variable, which is then returned in the `props` object.

It's important to understand that this function is part of the Node.js environment, where we can use server-side APIs. This means we can read files, access databases, and more. This provides significant capabilities for implementing complex full-stack projects.

Next, in the same `about.tsx` file, we have the `About` component:

```
export default function About({
  user,
}: InferGetServerSidePropsType<typeof getServerSideProps>) {
  return (
    <main>
      <Image src={user.avatar_url} alt={user.login} width="100" height="100" />
      <h2>{user.name || user.login}</h2>
      <p>{user.bio}</p>
      <p>Location: {user.location || "Not specified"}</p>
      <p>Company: {user.company || "Not specified"}</p>
      <p>Followers: {user.followers}</p>
      <p>Following: {user.following}</p>
      <p>Public Repos: {user.public_repos}</p>
    </main>
  );
}
```

In the `About` component, we use the `user` variable that we returned from the `getServerSideProps` function to create the page's markup. With just this one function, we've implemented SSR.

Next, let's create the `/posts` and `/posts/[post]` pages where we will implement SSG and ISR. For this, Next.js provides two functions: `getStaticProps` and `getStaticPaths`:

- `getStaticProps`: This function serves a similar purpose as `getServerSideProps` but is called during the project build process.

- `getStaticPaths`: This is used on dynamic pages where the path contains parameters (such as `[post].tsx`). This function determines which paths should be pre-generated during the build.

Let's take a look at how the `Posts` page component is implemented:

```
export async function getStaticProps() {
  const posts = ["1", "2", "3"];
  return {
    props: {
      posts,
    },
  };
}
export default function Posts({ posts }: { posts: string[] }) {
  return (
    <main>
      <h1>Posts</h1>
      <ul>
        {posts.map((post) => (
          <li key={post}>
            <Link href={` /posts/${post}`}>Post {post}</Link>
          </li>
        ))}
      </ul>
    </main>
  );
}
```

The `getStaticProps` function in this example doesn't request any data but simply returns three pages. However, just like in `getServerSide-`

Props, you can use `getStaticProps` to fetch data or work with the filesystem. The `Posts` component then receives posts as `props` and uses them to display a list of links to posts.

Here's what the **Posts** page will look like:



Figure 13.6: Posts page

When opening any post, the component from the `[post].tsx` file will be loaded. Here's how it looks:

```
export const getStaticPaths = (async () => {  
  return {  
    paths: [  
      {  
        params: {  
          post: "1",  
        },  
      },  
    ],  
  };  
})()
```

```

    },
  },
  {
    params: {
      post: "2",
    },
  },
  {
    params: {
      post: "3",
    },
  },
],
fallback: true,
};
}) satisfies GetStaticPaths;

```

This function informs the builder that only three pages need to be rendered during the build process. In this function, we can also make network requests. The "fallback" parameter we returned indicates that, theoretically, there may be more post pages than the ones we returned. For example, if we access the `/posts/4` page, it will be rendered in SSR mode and saved as the build result:

```

Export const getStaticProps = (async (context) => {
  const content = `This is a dynamic route example. The value of the post parameter is ${context.params?.post}.`;
  return { props: { content }, revalidate: 3600 };
}) satisfies GetStaticProps<{
  content: string;
}>;

```


In the `getStaticProps` function, we can now read the `page` parameter from the `context` argument. The value of `revalidate` that we returned from the function enables ISR and tells the server to rebuild this page on the next request, after `3600` from the previous build. Here's how the `Post` page will look:

```
export default function Post({
  content,
}: InferGetStaticPropsType<typeof getStaticProps>) {
  const router = useRouter();
  return (
    <main>
      <h1>Post - {router.query.post}</h1>
      <p>{content}</p>
    </main>
  );
}
```

When we open any post using the link, we will see the following:

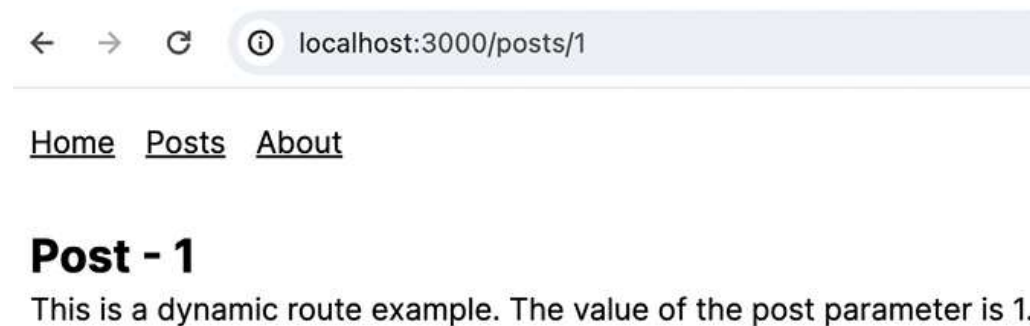


Figure 13.7: Post page

In this example, we've created a website where pages use different server rendering approaches, which is useful and convenient for building large and complex projects. However, Next.js has more capabilities beyond this. Next, we will explore a new approach to building websites using the App Router.

React Server Components

React Server Components represent a new paradigm for working with components in Next.js that eliminates isomorphic JavaScript. The code of such components runs only on a server and can be cached as a result. In this concept, you can directly read the server's filesystem or access the database from the components.

In Next.js, React Server Components allow you to categorize components into two types: **server-side** and **client-side**. Server-side components are processed on a server and sent to the client as static HTML, reducing the load on the browser. Client-side components still have all the capabilities of browser JavaScript but with one requirement: you need to use the `use client` directive at the beginning of the file.

To use server-side components in Next.js, you will need to create a new project. For routing, you still use files, but now, the main folder for the project is the `app` folder, and route names are based solely on folder names. Inside each route (folder), there should be files with names specified by the framework. Here are some of the key files:

- `page.tsx`: This file and its component will be used to display the page.

- `loading.tsx`: The component of this file will be sent to the client as a loading state while the component from the `page.tsx` file is executed and loaded.
- `layout.tsx`: This is equivalent to the `_app.tsx` file, but in this case, we can have multiple layouts that can be nested within each other in nested routes.
- `route.tsx`: This file is used to implement an API endpoint.

Now, let's refactor our website with posts using the new architecture based on the **App Router**. Let's start with the home page. Since our website didn't have any interactive elements, I suggest adding one. Let's create the simplest button with a counter and place it on the home page. Here's the code for such a button:

```
"use client";
import React from "react";
export const Counter = () => {
  const [count, setCount] = React.useState(0);
  return <button onClick={() => setCount(count + 1)}>{count}</button>;
};
```

This component renders a button with a counter inside. By clicking the button, we update the counter. To make this component work with App Router, we need to add the `"use client"` directive, which tells Next.js to include this component's code in the bundle and send it to the browser upon request.

Now, let's add this button to the home page, and here's what its code will look like:

```
export default function Home() {  
  return (  
    <main>  
      <h1>Home Page</h1>  
      <Counter />  
    </main>  
  );  
}
```

Since the page is simple, it doesn't differ from what we saw in the Pages Router, except for the new button. Although, by default, the App Router considers all components as server ones, in this case, the page will be rendered during the build process and saved as a static page.

Now, let's move on to the `About` page. To create this page, we need to create a folder named `about` and create a file inside it named `page.tsx`, where we'll place the component. Here's the code for it:

```
export const dynamic = "force-dynamic";  
export default async function About() {  
  const res = await fetch("https://api.github.com/users/sakhnyuk");  
  const user: GitHubUser = await res.json();  
  return (  
    <main>  
      <Image src={user.avatar_url} alt={user.login} width="100" height="100" />  
      <h2>{user.name || user.login}</h2>  
      <p>{user.bio}</p>  
      <p>Location: {user.location || "Not specified"}</p>  
      <p>Company: {user.company || "Not specified"}</p>  
      <p>Followers: {user.followers}</p>  
    </main>  
  );  
}
```

```
    <p>Following: {user.following}</p>
    <p>Public Repos: {user.public_repos}</p>
  </main>
);
}
```

As you can see, the code for this page has become simpler compared to using the Pages Router. The `About` component has become asynchronous, allowing us to make a network request and wait for the result. Since, in our example, we wanted to use SSR and render the page on the server for each request, we needed to export the “dynamic” variable from the file with the `force-dynamic` value. This parameter explicitly tells Next.js that we want to generate a new page for each request. Otherwise, Next.js would have generated the page during the project build and saved the result as a static page (by using SSG).

However, it would be strange if the App Router simply repeated the previous functionality without offering anything new. If we create a `loading.tsx` file inside the `about` folder, when opening the `About` page, instead of waiting for the server to request information from GitHub and prepare the page, it will instantly serve the page with content from the `loading` file as a fallback. And as soon as the component from the `page.tsx` file is ready, the server will send it to the client to replace the `loading` component. This provides a significant performance advantage and improves the user experience.

Now, let’s move on to the `Posts` page. Create a `posts` folder and a `page.tsx` file inside it. Here’s how the updated code for the `/posts` page will look:

```
export default async function Posts() {  
  const posts = ["1", "2", "3"];  
  return (  
    <main>  
      <h1>Posts</h1>  
      <ul>  
        {posts.map((post) => (  
          <li key={post}>  
            <Link href={` /posts/${post}`}>Post {post}</Link>  
          </li>  
        ))}  
      </ul>  
    </main>  
  );  
}
```

Once again, the code has become very clean. Everything we needed to fetch before rendering the page can be obtained and created directly inside the component. In our example, we have hardcoded three pages that will be rendered as links.

To implement a `Post` page, inside the `posts` folder, you need to create a folder with the name `[post]` and create the `page.tsx` file inside it. Here's the code, which is now much cleaner and more readable:

```
export async function generateStaticParams() {  
  return [{ post: "1" }, { post: "2" }, { post: "3" }];  
}
```

Instead of using `getStaticPaths`, we provide Next.js with information about the list of static pages to generate during the project build using the `generateStaticParams` function. Then, we use props inside the component to display the page's content:

```
export const revalidate = 3600
export default async function Post({ params }: { params: { post: string } }) {
  return (
    <main>
      <h1>Post - {params.post}</h1>
      <p>
        This is a dynamic route example. The value of the post parameter is
        {params.post}.
      </p>
    </main>
  );
}
```

The content remains mostly unchanged. To activate ISR, all we need to do is export the `revalidate` variable from the file with the revalidation value in seconds.

In this example, we covered the fundamental approaches to building an application using React Server Components and the App Router in Next.js. The Page Router and App Router examples provided in this chapter do not cover all the possibilities of Next.js. For a deeper understanding of this framework, I recommend checking out the excellent documentation on its website: <https://nextjs.org/docs>.

Summary

In this chapter, we explored SSR in the context of React applications. We discussed approaches such as SSR, SSG, and ISR, learning the advantages and disadvantages of each approach.

Then, we learned how to apply these approaches in an application using Next.js and the Pages Router. Finally, we introduced a new technology called React Server Components and the updated Next.js architecture called the App Router.

In the next chapter, we will learn how to test our components and applications.